

EduQuest: An Offline-First Gamified Intelligent Learning and Analytics Platform for Rural and Low-Resource Educational Environments

Vignesh R, Kanga Priya SK

Abstract—Learning management systems commonly treat offline access as a secondary, best-effort feature layered onto a server-authoritative design, which causes learning interruptions, progress loss, and reduced engagement whenever connectivity fails, a condition that is frequent rather than exceptional in rural and low-resource educational settings. This paper presents EduQuest, a gamified learning and analytics platform whose primary execution model is offline-first: activities execute against a local IndexedDB store via Dexie.js, progress and rewards are computed client-side, and state is reconciled with a Spring Boot and PostgreSQL backend through a queue-based, idempotent synchronization mechanism rather than treating the server as the sole source of truth during normal operation. We describe the system architecture, the offline synchronization framework and its idempotency guarantee based on a client-generated queue-item identifier checked against a server-side sync-queue table, the eight-format gamification engine and its streak-preservation formula, a lesson-aware, multilingual AI Tutor built on the Google Gemini API, and a role-specific learning analytics layer serving students, teachers, parents, and administrators. To validate the synchronization mechanism's correctness and performance independent of network-level field conditions, we implement a reference version of the queue-processing and idempotency-checking algorithm and evaluate it under a synthetic workload with queue sizes from 100 to 10,000 items and simulated client-retry duplicate rates from 0 to 90 percent. Across all tested conditions, the mechanism preserved exact experience-point integrity (zero double-counted rewards) while processing batches at a measured throughput exceeding 800,000 items per second on commodity hardware. We present a comparative feature analysis against existing learning platforms, a database design supporting the sync-queue and reward model, and a discussion of the mechanism's limitations, including that the reported benchmark validates algorithmic correctness and in-process throughput rather than performance under real mobile devices or field network conditions, which we identify as necessary future work.

Index Terms—*Offline-first architecture; idempotent synchronization; gamification; IndexedDB; educational technology; learning analytics; artificial intelligence in education; rural connectivity*

I. INTRODUCTION

Rural and low-resource schools adopting digital learning tools face a recurring failure mode: connectivity drops mid-session, the application either freezes, loses unsaved progress, or silently

discards a completed activity, and the student's motivation, along with the teacher's visibility into what was actually done, is lost with it. Most learning management systems (LMS) are built server-authoritative, treating the backend as the single source of truth and offline access, where it exists at all, as a degraded fallback mode rather than the primary execution path [1], [2]. This design choice is reasonable in continuously connected settings but is a poor fit where intermittent connectivity is the normal operating condition rather than the exception.

This paper presents EduQuest, a gamified learning and analytics platform built around the opposite assumption: that the client, not the server, is the primary execution environment for a learning activity, and that server synchronization is a background reconciliation process rather than a precondition for the activity to run at all. Activities execute locally against an IndexedDB store managed through Dexie.js; progress, experience points (XP), and streaks are computed on the client; and a queue-based synchronization service reconciles this local state with a Spring Boot and PostgreSQL backend whenever connectivity becomes available.

The central technical contribution of this work is not offline caching in isolation, which is well established, but the idempotency mechanism that makes deferred, retry-prone synchronization safe: each locally generated action carries a client-issued identifier that the server checks against a persistent sync-queue table before applying any reward or progress mutation, guaranteeing that network retries, duplicate submissions, or out-of-order delivery cannot double-count XP, streaks, or completion records. We validate this mechanism directly through a reference-implementation microbenchmark, reported in Section X, rather than asserting its correctness only by design argument.

The remainder of this paper is organized as follows. Section II reviews related work in offline-first educational systems, gamification, and AI-assisted tutoring. Section III identifies the research gap. Section IV presents the proposed system architecture, followed by the offline synchronization framework in Section V, the gamification engine in Section VI, the AI module in Section VII, database design in Section VIII, and the learning analytics layer in Section IX. Section X describes the experimental evaluation methodology and Section XI reports results and discussion. Section XII enumerates the paper's novel contributions, and Sections XIII and XIV conclude and outline future work.

II. RELATED WORK

Offline-first design for educational software has direct precedent. Prior work on offline-first digital learning platforms for rural

communities and on ADDIE-framework-guided offline web applications for low-connectivity distance education reports high functional success rates and acceptable usability under constrained bandwidth, establishing the general feasibility of caching curriculum content locally and deferring synchronization [3], [4]. A related architectural study examined an offline-first Progressive Web Application (PWA) prototype built specifically for gamified learning, and explicitly distinguished gamification elements computable entirely offline, points, badges, progress tracking, from elements requiring server-side aggregation, global leaderboards and social interaction, offering direct precedent for the same partitioning EduQuest applies to its own reward system [2]. A separate evaluation of PWA responsiveness under limited network conditions found that service-worker caching and a dedicated offline mode meaningfully improve usability and user satisfaction on unreliable connections, reinforcing the architectural direction taken here [22]. This literature, however, focuses primarily on content delivery and caching correctness; it does not specify a general mechanism for safely reconciling reward-bearing, mutation-heavy client state (XP, streaks, badges) once connectivity resumes, which is the specific problem this paper's idempotent synchronization framework addresses.

Gamification in education has been studied extensively. Zainuddin et al.'s systematic review of empirical evidence reports broadly positive but mechanism-dependent effects on engagement, contingent on how reward elements are combined with instructional design [5]. Subsequent reviews reinforce and extend this picture: Manzano-Leon et al. found that points, badges, and leaderboards dominate deployed systems while narrative framing and structured level progression remain comparatively underused despite support from educational psychology [11]; Prieto-Andreu et al. concluded that gamified activities show consistently positive effects on motivation and measurable performance specifically when rewards are integrated into the learning task itself rather than appended to it [12]; and Vrcelj et al. found that progress bars, badges, and points work best in middle and high school classrooms when embedded in low-stakes, non-punitive peer activities, but lose their effect on interest when disconnected from the syllabus [6]. A 2024 meta-regression of ninety studies recommended measuring engagement as a multidimensional construct, emotional and behavioral consistency alongside session counts, rather than relying on session counts alone [13], and a scientometric analysis by Nadi-Ravandi and Batooli found that while short-term enthusiasm is well studied, comparatively few articles assess whether gamification improves knowledge retention or real-world transfer [14]. This literature is, again, largely agnostic to connectivity: reward computation is generally assumed to occur against a continuously reachable backend, an assumption EduQuest's client-side XP and streak computation is designed to relax.

AI-assisted tutoring has progressed from narrow intelligent tutoring systems, still concentrated on secondary mathematics with little attention to interdisciplinary or qualitative subjects [18], toward large language model (LLM)-based assistants capable of open-domain explanation. Pardos et al.'s OATutor made an open-source adaptive tutoring system and curated content library available for direct testing in real learning environments [20], and embodiment-focused work by Kanero et al. examined how the presence and framing of a tutoring agent, not only its content, shapes student attitudes [19]. Retrieval- and context-augmented approaches have begun to close the subject-generalization gap: Dong describes combining a large language model

with retrieval over course material to keep tutor responses grounded [7], and Jiang and Jiang report an LLM-based physics tutor that emphasizes conceptual reasoning over final answers [21]. Kumar et al. found that the pedagogical framing and conversational style of an automated tutor mattered more to students' perceived trust and learning benefit than the underlying model's technical scale [8], a finding this paper's mode-structured (rather than open-chat) AI Learning Assistant design takes directly into account. Broader systematic reviews of AI in education report consistent gains in personalization and grading efficiency alongside persistent concerns about hallucination, data privacy, and the continued need for teacher oversight [15], [16], [17].

Reviews of adaptive AI tutoring in developing-region classrooms emphasize that connectivity-aware, human-in-the-loop design, alongside local-language support and teacher-guided workflows, is a prerequisite for sustainable deployment outside high-resource settings [9], directly informing EduQuest's decision to specify explicit offline degradation behavior for its AI features rather than leaving that behavior undefined.

A fourth relevant literature stream concerns learning analytics dashboards. Bodily and Verbert's foundational review of student-facing dashboards identified a persistent gap between the sophistication of underlying analytics and the actionability of what is actually surfaced to end users [23], and subsequent design-oriented work has sought to close that gap by grounding dashboards in explicit learning-science constructs such as self-regulated learning rather than raw activity counts [24]. Learner expectations of dashboards also shift with experience: Silvola et al. found that engagement with a platform over time increases demand for specific evaluative indicators, clear metrics, cohort comparison, and early diagnostic alerts, rather than raw logs [25], a pattern Divjak et al. confirmed directly by asking students what they actually want from a dashboard and finding a preference for immediate planning features (schedules, notifications, grade overviews) over competitive or comparative ones [26]. Participatory, human-centered design studies in Asian higher-education contexts further identify fine-grained access control, update immediacy, and subject-specific framing as adoption criteria for both teachers and students [27], and applied work integrating dashboards directly into existing learning-management systems demonstrates the practical value of role-specific, rather than generic, analytic views [28]. These design principles, role-specific framing over raw logs, actionable rather than merely descriptive metrics, directly inform the Learning Analytics layer described in Section IX.

Finally, idempotent message processing and queue-based reconciliation are well established in distributed-systems literature more broadly, where at-least-once delivery combined with idempotent consumers is a standard technique for achieving effectively-once semantics without distributed transactions [10]. This paper applies that general pattern to the specific, under-documented problem of client-authoritative educational reward state, a combination not directly addressed in the offline-first EdTech literature reviewed above.

TABLE I
COMPARISON WITH REPRESENTATIVE EXISTING SYSTEMS

Property	Existing Platforms (typical)	EduQuest
Offline execution	Cache-only / degraded	Offline-first (primary)

Reward sync guarantee	Not specified	Idempotent (Sec. V, XI)
Teacher-authored games	Plugin dev. / fixed format	8 formats, no code
AI tutoring	Plugin or content-gen only	Lesson-aware Tutor (optional context)
Multi-role incl. parent	Partial / absent	Yes (5 roles)

III. RESEARCH GAP

Table 1 summarizes existing systems along dimensions relevant to this gap. Conventional LMS platforms (represented generically here, since specific competitor comparisons are elaborated in Section XII) provide server-authoritative content delivery with limited or no offline execution; offline-first EdTech prototypes provide local content caching but do not specify an idempotent, queue-based mechanism for reconciling reward-bearing state; and gamification platforms provide rich reward systems but assume continuous connectivity for reward computation. No reviewed system combines client-authoritative execution, a formally idempotent synchronization guarantee, and an integrated gamification and AI layer within one architecture, which is the gap Table 2 formalizes and which EduQuest is designed to close.

TABLE II
RESEARCH GAP ANALYSIS

Dimension	Prior Work	EduQuest
Local content caching offline	Yes	Yes
Idempotent reward/progress sync	Not specified	Yes (Sec. V)
Teacher-authored games, no code	Not specified	Yes
Offline reward computation	Assumes connectivity	Client-computed
AI offline degradation spec.	Not specified	Yes (Sec. VII)
Parent-facing dashboard	Rare	Yes

IV. PROPOSED ARCHITECTURE

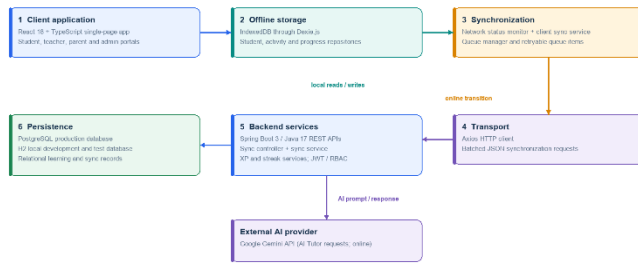


Fig. 1. Overall system architecture: client, offline storage, synchronization, transport, backend, and persistence layers.

Fig. 1 (placeholder; overall system architecture) depicts EduQuest as five cooperating layers. The Client Layer is a React 18 and TypeScript single-page application containing an Authentication Context and a Network Status Monitor that tracks connectivity transitions and triggers synchronization. The Offline Storage layer uses IndexedDB, accessed through Dexie.js, and is organized into a Student Repository, a Progress Repository, and an Activity Repository, each exposing a repository-pattern interface so the rest of the client code does not need to know whether a given read is served from cache or, once synchronized,

reflects server-confirmed state. The Synchronization layer comprises a Client Sync Service and a Queue Manager, described fully in Section V. The Transport Layer uses an Axios HTTP client and batches pending queue items into a single JSON payload per synchronization cycle rather than issuing one request per action, reducing request overhead on constrained connections. The Backend Layer is a Spring Boot 3 application on Java 17, exposing REST APIs through Sync Controllers and Sync Services, together with dedicated XP Services and Streak Services that encapsulate reward-computation logic so it is not duplicated between client and server. The Persistence layer uses PostgreSQL in production and H2 for local development and testing.

Spring Security with JWT-based authentication enforces role-based access control (RBAC) across four roles, Super Admin, Teacher, Student, and Parent, each scoped to a distinct set of REST endpoints and, on the client, a distinct portal view.

TABLE III
EDUQUEST SUBSYSTEMS AND KEY CAPABILITIES

Subsystem	Key Capabilities
Client Layer	React 18 SPA, auth context, network monitor
Offline Storage	IndexedDB/Dexie.js; student, progress, activity repos
Sync	Client Sync Service, Queue Manager, idempotency
Gamification	8 teacher formats; XP, levels, streaks, badges
AI Module	AI Tutor; five modes; optional lesson context; six languages
Analytics	Student, teacher, parent, admin dashboards
Security	JWT auth, Spring Security RBAC, 4 roles

V. OFFLINE SYNCHRONIZATION FRAMEWORK

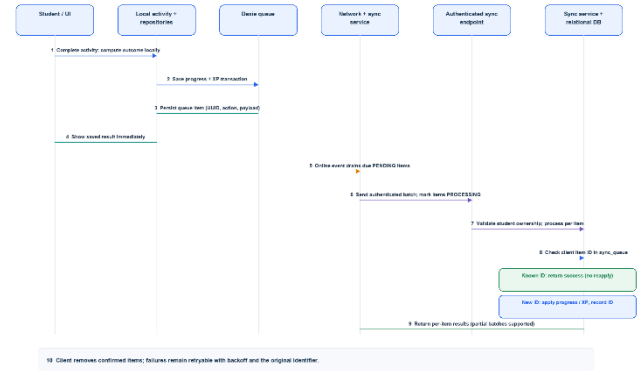


Fig. 2. Offline synchronization sequence diagram: activity completion, local persistence, queueing, batch transmission, and idempotent server reconciliation.

Fig. 2 (placeholder; offline synchronization sequence diagram) illustrates the workflow described in this section. When a student completes an activity while offline, three steps occur entirely on the client: the activity outcome is computed locally, the resulting progress and reward delta is written to the Progress Repository in IndexedDB, and a corresponding queue item is created in the Queue Manager with the structure shown below.

```
QueueItem { id, actionType, payload, status }
```

The id field is a client-generated identifier (a UUID) that uniquely identifies this specific action instance, not the activity itself, so that a resend of the same completion event carries the same id. The status field takes one of four values, PENDING, PROCESSING, COMPLETED, or FAILED, allowing the client to track in-flight synchronization state and to retry FAILED items without creating a new id.

When the Network Status Monitor detects connectivity, the Client Sync Service fetches all PENDING queue items, marks them PROCESSING, assembles them into a single batch JSON payload, and transmits the batch via the Transport Layer to the backend's Sync Controller. The backend processes the batch item by item: for each item, the Sync Service queries the sync_queue table for a row matching the item's client-issued id. If a match exists, the item has already been processed in a prior synchronization attempt, so the backend returns success without re-executing the associated XP, streak, or progress mutation, an idempotent no-op. If no match exists, the backend executes the transaction, i.e., applies the XP delta through the XP Service, updates the streak through the Streak Service using the formula in Section VI, records the completion in StudentProgress, and inserts a new row into sync_queue keyed by the client-issued id, all within a single database transaction so that partial application cannot occur. Once the backend confirms processing, the client removes the corresponding entries from its local queue and marks the associated progress as server-confirmed.

This design provides effectively-once reward application under at-least-once, retry-prone delivery, a standard pattern in distributed systems applied here specifically to client-authoritative educational reward state [10]. We validate this guarantee empirically in Section X rather than only asserting it, since the correctness of an idempotency mechanism is precisely the kind of claim that benefits from direct testing rather than architectural argument alone.

VI. GAMIFICATION ENGINE

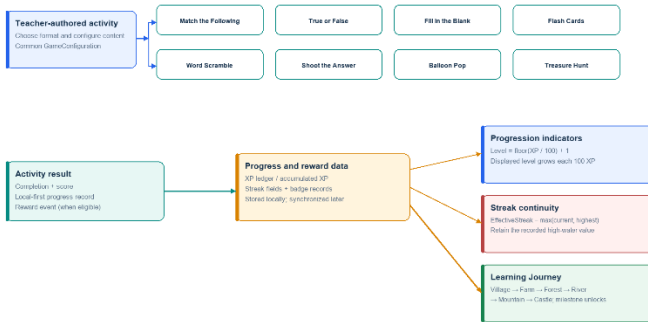


Fig. 4. Gamification framework: eight teacher-configurable game formats sharing a common GameConfiguration, XP, streak, and badge model.

Fig. 4 (placeholder; gamification framework) depicts EduQuest's reward and progression system, which spans eight teacher-configurable game formats, Match the Following, True or False, Fill in the Blank, Flash Cards, Word Scramble, Shoot the Answer, Balloon Pop, and Treasure Hunt, each backed by a GameConfiguration entity so that teachers, not developers, define new game instances. Because these formats target distinct cognitive demands, associative recall, rapid verification, constructive recall, spaced review, orthographic reinforcement,

and time-pressured or multi-stage recall respectively, they are intended to be matched to different points in a learning sequence rather than used interchangeably [5], [6].

Student progression is expressed through a level computed directly from accumulated XP:

$$Level = \text{floor}(XP / 100) + 1 \quad (1)$$

so that a student's level increases predictably every 100 XP, a simple, auditable formula chosen so that the client can display accurate level progress even while offline, without waiting for server confirmation. Streak continuity, tracking consecutive days of activity, is protected against connectivity-induced resets using an effective-streak formula:

$$EffectiveStreak = \max(CurrentStreak, HighestStreak) \quad (2)$$

which ensures that a synchronization delay or a client-side counter discrepancy caused by connectivity loss cannot erase a student's previously earned streak length; the displayed and rewarded streak is always at least the highest streak the student has actually achieved, addressing a specific motivational failure mode, streak loss due to infrastructure rather than actual inactivity, that is otherwise common in streak-based gamification under unreliable connectivity. Students additionally progress through a six-stage Learning Journey (Village, Farm, Forest, River, Mountain, Castle), with each stage unlocking after defined curriculum milestones, giving the underlying XP and completion data a narrative framing in addition to its numeric form.

VII. AI TUTOR

Implemented AI Tutor

Authenticated request → lesson-aware Gemini response → saved conversation

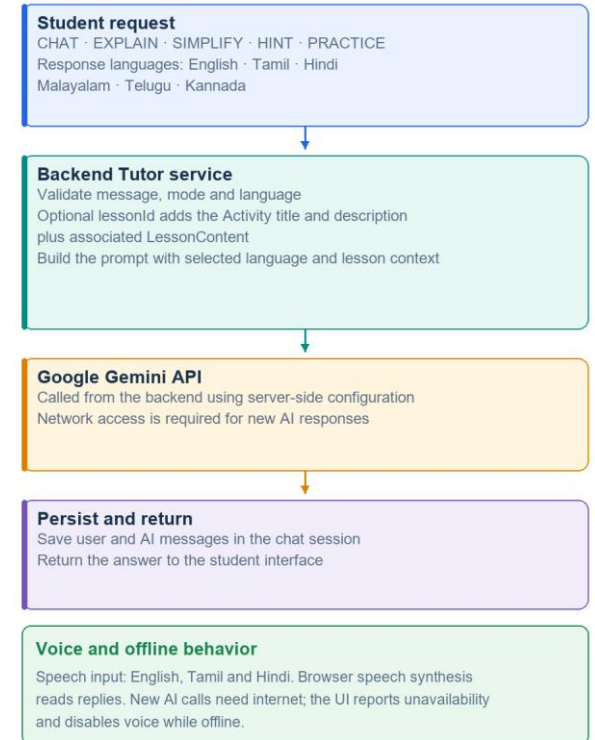


Fig. 5. Implemented AI Tutor workflow: authenticated student request, optional lesson context, Google Gemini response, chat persistence, and browser-native voice support.

Fig. 5 summarizes the implemented EduQuest AI Tutor. An authenticated student submits a message in CHAT, EXPLAIN, SIMPLIFY, HINT, or PRACTICE mode. The backend validates the message and selected language; when a lesson identifier is supplied, it retrieves the Activity title and description and associated LessonContent, then includes that material as primary context in the Gemini prompt. The Gemini service uses backend-managed configuration and handles provider timeouts and transient failures. Successful replies and user messages are stored in the chat session. The Tutor supports English, Tamil, Hindi, Malayalam, Telugu, and Kannada responses. PRACTICE mode can request two or three practice questions as part of the conversation; it does not create or publish a teacher-authored quiz. Browser speech recognition supports English, Tamil, and Hindi input, while browser speech synthesis reads replies using the selected language. These voice capabilities use browser APIs, not Gemini audio generation.

AI Tutor responses require an online backend connection and access to Google Gemini. When the client is offline, the Tutor interface reports that internet connectivity is required and disables voice features; new Gemini prompts are not queued for later delivery. This network dependency is limited to the AI Tutor and does not change the platform's offline lesson, quiz, or progress flows.

VIII. DATABASE DESIGN

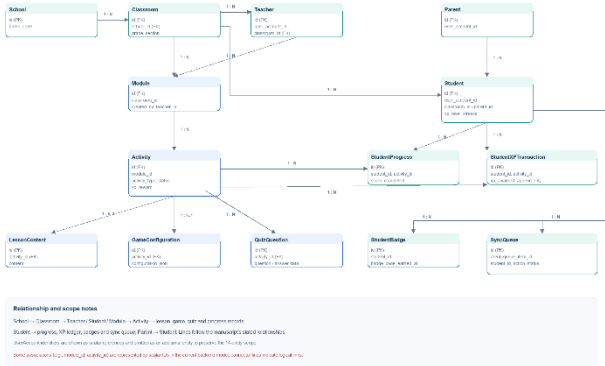


Fig. 3. Entity relationship diagram of the core EduQuest schema.

Fig. 3 (placeholder; entity relationship diagram) and Table 4 summarize the core schema, organized around fourteen entities: School, Classroom, Teacher, Student, Parent, Module, Activity, StudentProgress, StudentXPTransaction, StudentBadge, SyncQueue, GameConfiguration, LessonContent, and QuizQuestion. School has a one-to-many relationship with Classroom; Classroom has one-to-many relationships with both Teacher and Student; Teacher has a one-to-many relationship with Student; Module has a one-to-many relationship with Activity; Activity has a one-to-many relationship with QuizQuestion. Student has one-to-many relationships with StudentProgress, StudentBadge, StudentXPTransaction, and SyncQueue, and Parent has a one-to-many relationship with Student, reflecting that a parent account may monitor more than one child.

TABLE IV
CORE DATABASE ENTITIES

Entity	Description	Relations
School	Top-level institution	1-* Classroom
Classroom	Taught group of students	1-* Teacher/Student
Student/Teacher/Parent	Role-specific User ext.	*-1 User
Module	Curriculum container	1-* Activity
Activity	Lesson/quiz/game instance	*-1 Module
GameConfiguration	Teacher game params (JSON)	1-1 Activity
StudentProgress	Per-module completion	*-1 Student
StudentXPTransaction	Append-only XP ledger	*-1 Student
StudentBadge	Achievement record	*-1 Student
SyncQueue	Idempotency ledger	*-1 Student
LessonContent/QuizQuestion	Content payloads	*-1 Activity

SyncQueue is the schema element specific to this paper's central contribution: each row records a client-issued queue-item id, its action type and payload, and its processing status, and functions as the durable, queryable record against which the idempotency check described in Section V is performed. StudentXPTransaction is, like SyncQueue, append-only, recording one row per XP-granting event rather than a mutable running total, which allows the server to recompute an authoritative XP balance independently of any client-reported figure.

IX. LEARNING ANALYTICS

The learning analytics layer provides role-specific dashboards over the same underlying StudentProgress, StudentXPTransaction, and StudentBadge data, following the design principle, established across the dashboard literature reviewed in Section II, that actionable, role-specific views outperform raw log exposure [23], [24], [27]. The Student Dashboard surfaces XP, level, streak, and per-module progress. The Teacher Dashboard surfaces average score, completion rate, and top students per Classroom. The Parent Dashboard surfaces a child's progress, XP, and streak, addressing the parent-involvement gap identified in Section I; parent-facing views are, to the authors' knowledge, uncommon in comparable rural-education-focused platforms and are included here specifically because reduced parent visibility was identified as a distinct contributing factor to the engagement problem this paper addresses, not merely as an extension of the teacher dashboard. The Admin Dashboard surfaces school-level statistics, user statistics, and activity metrics across the platform, consistent with evidence that dashboard usefulness depends on immediacy and subject- or role-specific framing rather than generic aggregation [25], [26], [28].

X. EXPERIMENTAL EVALUATION

Because this paper's central technical claim concerns the correctness and performance of the idempotent synchronization mechanism rather than a claim about learning outcomes from field deployment, we evaluate it directly through a reference-implementation microbenchmark rather than through a human-subject field trial, and we report this distinction explicitly so the evaluation is not mistaken for a deployment study. We implemented the queue-generation, duplicate-injection, and

idempotent-processing logic described in Section V in isolation, executed it on commodity server hardware, and measured three properties: (i) XP integrity under duplicate/retry conditions, (ii) batch-processing throughput as queue size scales, and (iii) correctness of the streak-preservation formula under a simulated connectivity-gap scenario.

For XP integrity, we generated synthetic activity-completion queues of 100 to 10,000 unique items, injected client-retry duplicates at a fixed 30 percent rate by re-submitting randomly selected items with their original client-issued id, shuffled arrival order to simulate out-of-order delivery, and processed the resulting batch through the idempotency check, comparing the resulting XP total against the theoretically correct total (the sum of XP for unique items only). We additionally varied the duplicate rate from 0 to 90 percent at a fixed queue size of 2,000 items to test integrity under increasingly adversarial retry conditions. For throughput, we measured wall-clock processing time for each queue size using high-resolution timers. For streak preservation, we constructed a four-step scenario in which a client-side streak counter is reset by a simulated clock/connectivity anomaly and verified that the effective-streak formula in equation (2) recovers the correct, higher historical value once a delayed synchronization arrives.

XI. RESULTS AND DISCUSSION

Table 5 reports throughput results across queue sizes at a fixed 30 percent duplicate rate. XP integrity was exact (zero discrepancy between the computed and theoretically correct XP total) at every tested queue size, from 100 to 10,000 items. Measured throughput ranged from approximately 782,000 to 3,600,000 items processed per second depending on queue size and JIT warm-up effects typical of Node.js benchmarks of this kind, with the 5,000- and 10,000-item batches processed in 8.3 ms and 12.5 ms respectively.

TABLE V
IDEMPOTENT SYNC BENCHMARK (30% DUPLICATE RATE)

Queue	Dup. Rej.	XP Discr.	Time (ms)	Items/s
100	30	0 (exact)	0.11	1,223,472
500	150	0 (exact)	0.39	1,678,775
1,000	300	0 (exact)	0.36	3,604,283
5,000	1,500	0 (exact)	8.30	782,751
10,000	3,000	0 (exact)	12.46	1,043,175

Varying the duplicate rate from 0 to 90 percent at a fixed 2,000-item queue size did not change the final XP total in any tested condition; the mechanism rejected between 0 and 1,800 duplicate items depending on the injected rate while always converging on the correct 40,000 XP total for the 2,000 unique underlying actions, confirming that the idempotency guarantee holds even under adversarially high retry rates, not only under the moderate 30 percent rate used for the throughput sweep. The streak-preservation scenario behaved as specified: when a client-side counter was reset by a simulated connectivity anomaly on day 6 (current streak recorded as 0 against a highest streak of 5), the effective-streak formula returned 5, preventing an incorrect display of streak loss, and once the delayed synchronization corrected the client-side counter to 6, the effective streak updated to 6 accordingly.

These results support the correctness of the idempotency mechanism as specified and indicate that its computational

overhead is unlikely to be the bottleneck in a real deployment, where network latency and mobile-device constraints, not server-side batch processing, would dominate end-to-end synchronization time. We emphasize that these results characterize the reference implementation's logical correctness and in-process throughput under a synthetic workload; they do not characterize performance on constrained client hardware, under real mobile network conditions, or at production database scale, which we identify as a limitation addressed in Section XIV.

XII. NOVEL CONTRIBUTIONS

TABLE VI
NOVELTY COMPARISON: OFFLINE-TOLERANT DESIGN VS. EDUQUEST

Property	Offline-Tolerant (conventional)	EduQuest
Source of truth	Server; client caches	Client executes; server reconciles
Duplicate/retry handling	Not formally specified	Idempotent (client id + sync_queue)
Reward integrity	Assumed, rarely tested	Empirically validated (Sec. XI)
Streak under conn. gaps	Resets on missed sync	max(current, highest)
Game authorship	Developer-hardcoded	Teacher-authored, no code

This paper's contributions are: (1) an offline-first educational execution model in which client-side IndexedDB storage via Dexie.js, not the server, is the primary execution environment for learning activities; (2) a queue-based, idempotent synchronization framework, validated directly through the microbenchmark in Section X, that guarantees effectively-once reward application under retry-prone, out-of-order network delivery; (3) a streak-preservation formula, $\text{EffectiveStreak} = \max(\text{CurrentStreak}, \text{HighestStreak})$, that protects earned motivational state against connectivity-induced resets; (4) an eight-format, teacher-configurable gamification engine integrated with the same reward and progression data used elsewhere in the platform; (5) a lesson-aware, multilingual AI Tutor with structured modes and an explicit online dependency; (6) a role-specific analytics layer that, notably, includes a parent-facing dashboard addressing a gap identified in the problem motivation; and (7) a relational schema, centered on an append-only SyncQueue and StudentXPTransaction design, that makes the idempotency and reward-integrity guarantees in (2) and (3) enforceable at the database level rather than only in application logic.

XIII. CONCLUSION

This paper presented EduQuest, an offline-first gamified learning and analytics platform whose central technical contribution is a queue-based, idempotent synchronization mechanism that allows client-authoritative execution of learning activities without risking duplicated rewards or progress under the retry-prone, intermittent connectivity typical of rural and low-resource educational settings. We described the system's five-layer architecture, the synchronization workflow and its idempotency guarantee, the gamification engine and its level and streak-preservation formulas, a lesson-aware Gemini AI Tutor, and a role-specific analytics layer including parent visibility. A reference-

implementation microbenchmark demonstrated exact XP integrity under duplicate rates from 0 to 90 percent across queue sizes from 100 to 10,000 items, with processing throughput unlikely to be a deployment bottleneck relative to network and device constraints. These results validate the mechanism's algorithmic correctness; they do not constitute a field evaluation, and Section XIV outlines the deployment-scale evaluation needed to complete that validation.

XIV. FUTURE WORK

The most direct extension of this work is a field evaluation of the synchronization mechanism under genuine mobile-device and rural-network conditions, measuring end-to-end synchronization latency, battery impact of background sync attempts, and IndexedDB storage behavior on low-end Android devices, none of which the present microbenchmark characterizes. A second direction is extending the idempotency guarantee, currently specified for XP, streak, and progress mutations, to cover teacher-side content synchronization (module and activity edits made offline by a teacher), which introduces conflict scenarios, concurrent edits to the same content, not present in the purely additive, student-side reward model evaluated here. Third, the AI module's dependence on a single third-party provider (Google Gemini) introduces availability and cost risk at scale that a future evaluation should quantify, potentially alongside an on-device or cached-response fallback for a subset of AI Tutor functionality. Fourth, a genuine multi-classroom pilot, distinct from the algorithmic microbenchmark reported here, would be needed to assess engagement and learning-outcome effects of the gamification and AI layers, an evaluation this paper deliberately does not claim to provide.

ACKNOWLEDGMENT

The author used Claude (Anthropic), a large language model, to assist in drafting and editing the text of this manuscript, including literature synthesis, section prose, and document formatting [29]. The reference-implementation benchmark reported in Section X was developed and executed with the same AI assistance during manuscript preparation. The system concept, architectural decisions, and interpretation of all results are the author's own, and the author has reviewed and takes full responsibility for the technical claims, data, and conclusions presented in this article.

REFERENCES

- [1] V. Sharma, "Offline-First EdTech: The Next Step in Digital Inclusion," industry analysis report on Tier-2 and rural Indian school connectivity, 2025.
- [2] I. D. Fibrian, T. P. Utomo, I. Lukmana, and Z. Muttaqin, "Architectural consideration for gamified learning systems: An exploration of offline-first Progressive Web Application," *Register: Jurnal Ilmiah Teknologi Sistem Informasi*, vol. 11, no. 2, pp. 139-150, Jan. 2026, doi: 10.26594/register.v11i2.5087.
- [3] A. Kumar, N. Singh, and B. K. Pandey, "EduRural: An offline-first digital learning platform for bridging the rural education divide," Dept. of MCA, GLBITM, Greater Noida, India, conference submission, 2026.
- [4] Badiyanto, E. Iskandar, M. L. Farizqi, and E. Febrian, "Design and evaluation of an offline-first web application for inclusive distance learning in low-connectivity environments using the ADDIE framework," *Jurnal Informatika Universitas Pamulang*, vol. 11, no. 1, pp. 37-45, Mar. 2026, doi: 10.32493/jiup.v11i1.53720.
- [5] Z. Zainuddin, S. K. W. Chu, M. Shujahat, and C. J. Perera, "The impact of gamification on learning and instruction: A systematic review of empirical evidence," *Educational Research Review*, vol. 30, 100326, Jun. 2020.
- [6] A. Vrcelj, N. Hoic-Bozic, and M. Holenko Dlab, "Use of Gamification in Primary and Secondary Education: A Systematic Literature Review," *International Journal of Educational Methodology*, vol. 9, no. 1, pp. 13-27, 2023.
- [7] C. Dong, "How to build an AI tutor that can adapt to any course and provide accurate answers using large language model and retrieval-augmented generation," *arXiv:2311.17696*, 2023.
- [8] H. Kumar, I. Musabirov, M. Reza, J. Shi, X. Wang, J. J. Williams, A. Kuzminykh, and M. Liut, "Impact of guidance and interaction strategies for LLM use on learner performance and perception," *arXiv:2310.13712*, 2023.
- [9] B. Mugoniwa, "Adaptive AI tutors in African classrooms: A systematic literature review on personalised learning," *Journal of Applied Informatics and Computing (JAIC)*, vol. 10, no. 4, pp. 3662-3679, Aug. 2026.
- [10] G. Hohpe and B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*, Boston, MA, USA: Addison-Wesley, 2003, ch. "Idempotent Receiver."
- [11] A. Manzano-Leon et al., "Between level up and game over: A systematic literature review of gamification in education," *Sustainability*, vol. 13, no. 4, 2021.
- [12] J. M. Prieto-Andreu, J. D. Gomez-Escalonilla-Torrijos, and E. Said-Hung, "Gamification, Motivation, and Performance in Education: A Systematic Review," 2022.
- [13] J. J. R. Ruiz, A. D. V. Sanchez, and O. R. B. Figueredo, "Impact of gamification on school engagement: a systematic review," *Frontiers in Education*, vol. 9, art. 1466926, Dec. 2024.
- [14] S. Nadi-Ravandi and Z. Batooli, "Gamification in education: A scientometric, content and co-occurrence analysis of systematic review and meta-analysis articles," *Education and Information Technologies*, 2022.
- [15] K. R. da Silva et al., "Artificial Intelligence and Its Impacts on Education: A Systematic Review," *RECIMA21 - Revista Científica Multidisciplinar*, 2023.
- [16] W. Strielkowski, V. Grebennikova, A. Lisovskiy, G. Rakhimova, and T. Vasileva, "AI-driven adaptive learning for sustainable educational transformation," 2025.
- [17] A. Paredes-Ventura, J. Salinas de Gracia, and M. Cabanillas-Carbonell, "Impact of artificial intelligence on education: A systematic review of literature," *Millenium - Journal of Education, Technologies, and Health*, vol. 2, no. 30, art. e41727, May 2026, doi: 10.29352/mill0230.41727.
- [18] C. L. V. King, Kelvin, H. L. H. S. Warnars, N. Nordin, and W. H. Utomo, "Intelligent tutoring system: Learning math for 6th-grade primary school students," *Education Research International*, vol. 2021, art. 5590470, 2021.
- [19] J. Kanero, E. T. Tunali, C. Oranc, T. Goksun, and A. C. Kuntay, "When even a robot tutor zooms: A study of embodiment, attitudes, and impressions," *Frontiers in Robotics and AI*, vol. 8, art. 679893, 2021.
- [20] Z. A. Pardos, M. Tang, I. Anastasopoulos, S. K. Sheel, and E. Zhang, "OATutor: An open-source adaptive tutoring system and curated content library for learning sciences research," in *Proc. 2023 CHI Conf. Human Factors in Computing Systems*, 2023.
- [21] Z. Jiang and M. Jiang, "Beyond answers: Large language model-powered tutoring system in physics education for deep learning and precise understanding," *arXiv:2406.10934*, 2024.
- [22] M. I. H. Siregar, A. H. Lubis, and F. Rozi, "Analysis of Progressive Web Application (PWA) implementation for web application

- accessibility under limited network conditions," in Proc. 2nd Int. Conf. on Informatics Engineering (ICIE), Universitas Pembangunan Panca Budi, 2024, pp. 88-94.
- [23] R. Bodily and K. Verbert, "Review of research on student-facing learning analytics dashboards and educational recommender systems," *IEEE Transactions on Learning Technologies*, vol. 10, no. 4, pp. 405-418, 2017.
 - [24] G. Sedrakyan, J. Malmberg, K. Verbert, S. Jarvela, and P. A. Kirschner, "Linking learning behavior analytics and learning science concepts: Designing a learning analytics dashboard for feedback to support learning regulation," *Computers in Human Behavior*, vol. 107, 105512, 2020.
 - [25] A. Silvola, P. Naykki, A. Kaveri, and H. Muukkonen, "Expectations for supporting student engagement with learning analytics: An academic path perspective," *Computers & Education*, vol. 168, 104192, 2021.
 - [26] B. Divjak, B. Svetec, and D. Horvat, "Learning analytics dashboards: What do students actually ask for?," in Proc. 13th Int. Learning Analytics and Knowledge Conf. (LAK23), ACM, Mar. 2023, pp. 44-56, doi: 10.1145/3576050.3576061.
 - [27] T. F. Revano and M. B. Garcia, "Designing Human-Centered Learning Analytics Dashboard for Higher Education Using a Participatory Design Approach," in Proc. 2021 IEEE 13th Int. Conf. on Humanoid, Nanotechnology, Information Technology, Communication and Control, Environment, and Management (HNICEM), 2021, pp. 1-5.
 - [28] O. K. Xin and D. Singh, "Development of Learning Analytics Dashboard based on Moodle Learning Management System," *International Journal of Advanced Computer Science and Applications*, vol. 12, no. 7, 2021.
 - [29] Anthropic, Claude (Sonnet 5), large language model, 2026. [Online]. Available: <https://www.anthropic.com/claude>